

DATA STRUCTURES

Code:- 246C1203

L-T-P-S:- 3-0-2-4

Syllabus:-

→ Algorithmic Analysis } CO-1
→ Sorting

→ Linked List
→ Stack Applications } CO-2
→ Queue types

→ Hashing
→ Tree
* Binary Search Tree
* AVL Tree
* Splay Tree
* Red Black Tree
* B-Tree } CO-3

→ Graph
→ Minimum Spanning Tree } CO-4
→ Kruskal's
→ BFS & DFS (Breadth First Search & Depth First Search)

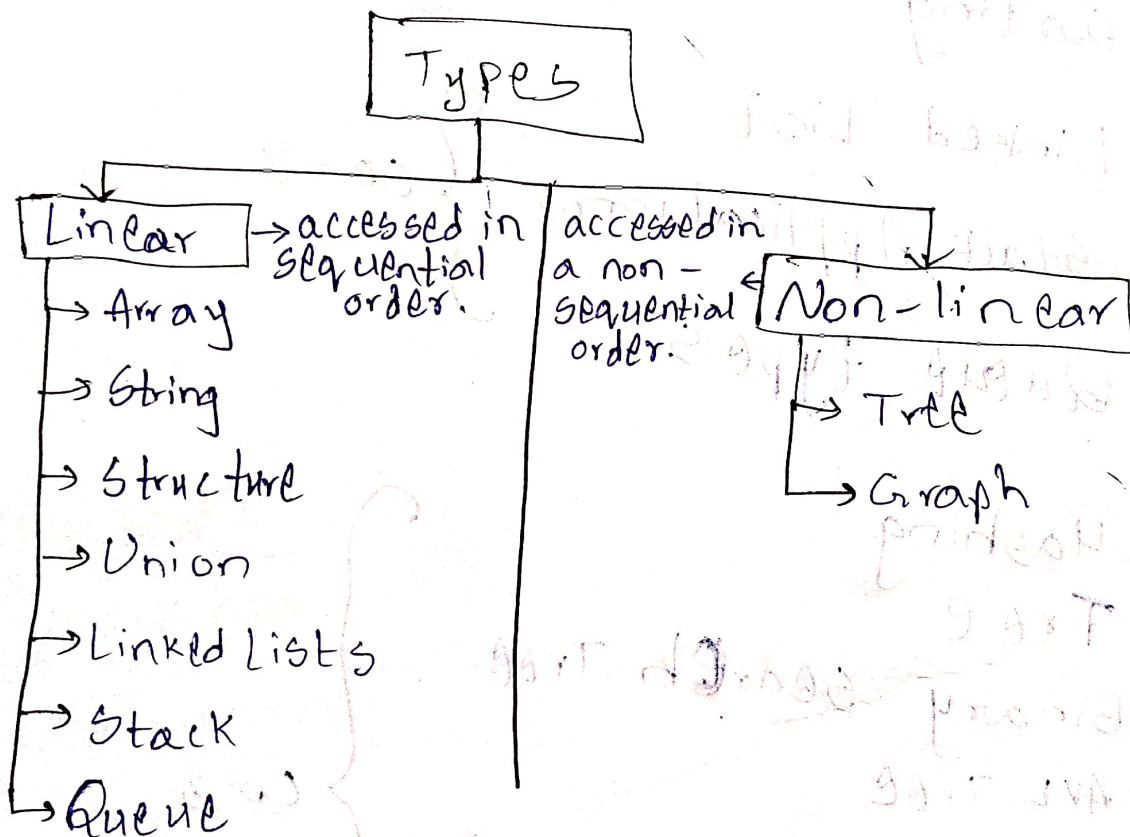
Data Structure:-

To organize data in efficient manner is called data structure.

(or)

To store, retrieve & manipulate data in efficient manner is called data structure.

Types of D.S



Algorithmic Analysis

* Asymptotic Notation is a mathematical tool to measure efficiency of the algorithm.

* Efficiency refer to $\left\{ \begin{array}{l} \rightarrow \text{Time Complexity} \\ \rightarrow \text{Space complexity} \end{array} \right.$ for an algorithm.

Time Complexity Analysis

* Three ways to measure Time complexity.

① Big O () → (Worst case of algorithm can be measured).

② Omega Ω () → (Best case)

③ Theta Θ () → (Average case)

Note:- An algorithm ~~should~~^{can} be measured in three parameters (Worst, Best, Average cases).

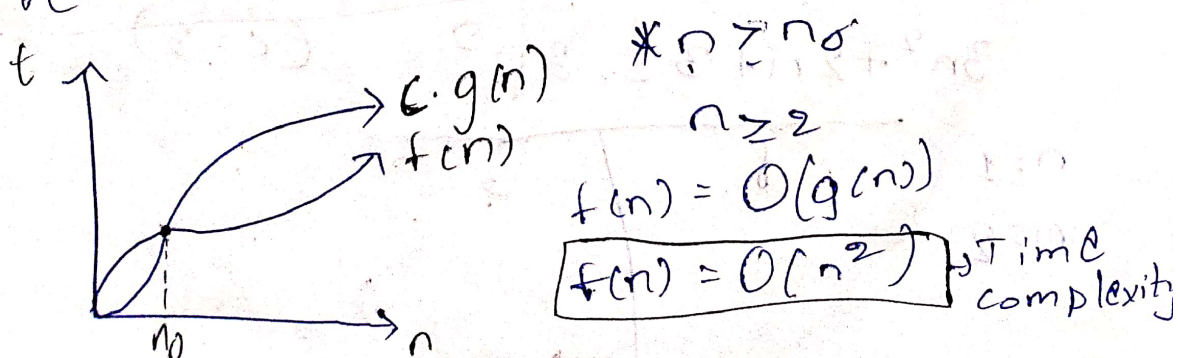
* Lower Bound $\leq$ Average Time $\leq$ Upper Bound
Big O ()

Consider $f(n)$ & $g(n)$ as a function.

* If $f(n) = O(g(n))$ is true, it means $f(n) \leq c * g(n)$ where 'c' is a constant.

Ex:- $n^2 + 3 \leq c \cdot n^2$ As n^2 is the highest polynomial function we take $g(n) = n^2$.

* $c=2$ as if we take $c=1$ condition will be false. Now it is true.

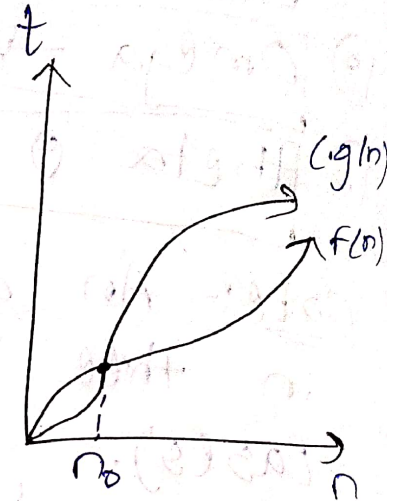


$3n^2 + 2n + 5 \leq c \cdot n^2$ As n increases $g(n)$ becomes the upper bound over $f(n)$.

$c = 1 \times$
 $= 2 \times$
 $= 3 \times$

$c = 4 \quad 3n^2 + 2n + 5 \leq 4 \cdot n^2$

$n = 1$	10	4
$n = 2$	21	16
$n = 3$	38	36
$n = 4$	61	64



$n \geq 4$

Time complexity $f(n) = O(g(n))$
 $f(n) = O(n^2)$

Omega Ω ()

24/12/24

The Omega notation can represent lower bound of an algorithm's time complexity.

$f(n) = \Omega(g(n))$

$f(n) \geq c \cdot g(n)$

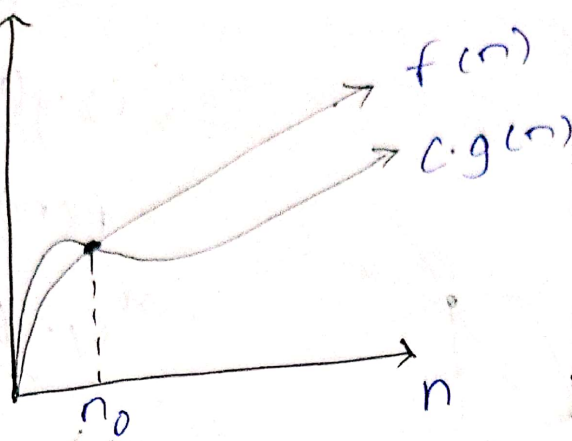
Ex:- $3n^2 + 2n + 3 \geq c \cdot n^2$

$3n^2 + 2n + 3 \geq 3n^2 \quad (c = 3)$

$n = 1$	8	3
$n = 2$		



* As 'n' increases $g(n)$ becomes lower bound below $f(n)$.



Time complexity $\Rightarrow f(n) = \Omega(n^2)$

theta $\Theta()$

The theta notation can represent lower bound and upper bound of an algorithm time complexity.

$$f(n) = \Theta(g(n))$$

$$* c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

$$3n^2 \leq 3n^2 + 2n + 3 \leq 4n^2$$

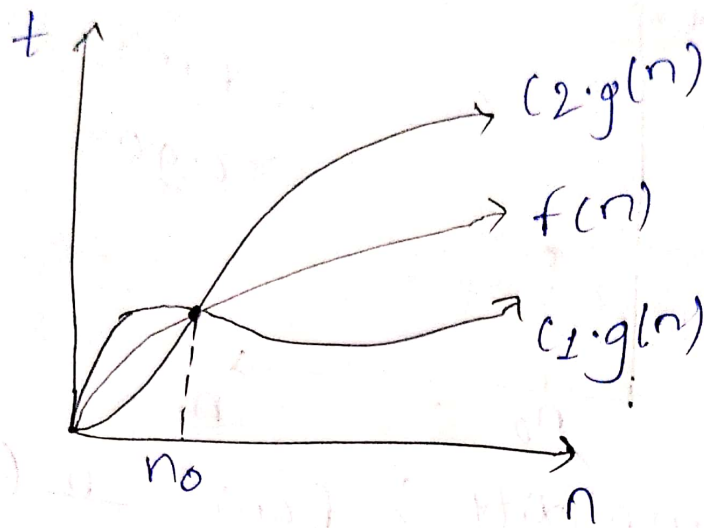
$$c_1 = 3$$

	3	8	4
$n=1$			
$n=2$		19	16
$n=3$		36	36

$n \geq 1 \Rightarrow c_1 \cdot g(n)$ is lower bound

$n \geq 3 \Rightarrow c_2 \cdot g(n)$ is upper bound





Q. $f(n) = 3n^3 + 2n^2 + 5n + 6$ & $g(n) = n^2$
 is $g(n)$ is upper bound of $f(n)$ or
 not?

A) $f(n) = O(g(n))$
 $f(n) \leq c \cdot g(n)$

$$3n^3 + 2n^2 + 5n + 6 \leq c \cdot n^2$$

* we could not find the constant 'c'
 value to prove that $g(n)$ is upper
 bound.

Ex:-

```
int main() → O(1)
int a=5, b=6, sum; O(1)
sum = a+b; O(1)
printf("%d", sum); O(1)
return 0; O(1)
}
```

* Total Time complexity $\Rightarrow O(1)$

Ex:- if (age >= 18)
 printf ("eligible to vote");
else
 printf ("Not eligible to vote");

Time complexity $\Rightarrow O(1)$

Ex:- for (i = 1; i <= n; i++)
 {
 sum = sum + i;
 }

$O(n)$

 printf ("sum = %d", sum); $\Rightarrow O(1)$

As $O(n) > O(1)$ the time complexity is $O(n)$.

Ex:- for (i = 1; i <= n; i = i + 2){
 sum = sum + i;
}
printf ("%d", sum);

* Here actually the complexity is $n/2$ but, while representing the time complexity constants are not allowed except 1.
So, $O(n)$. (or) $O(n/2) \cong O(n)$

Ex:- for($i=1$; $i \leq n$; $i = i * 2$) {
 sum = sum + i;

} printf("%d", sum);

1 2 4 8 16 32

G

$$2^k = n$$

$$k \approx 3$$

A;

$$n \approx 2^k$$

$$\log n \approx k$$

Time complexity $O(\log n)$ for division
 also same complexity i.e. $i = n$; $i > 1$
 $i = i / 2$

Ex:- for($i=1$; $i \leq n$; $i++$) { $\Rightarrow O(n)$
 for($j=1$; $j \leq n$; $j++$) { $\Rightarrow O(n)$
 printf("%d", i+j);

E

}

}

Time complexity $\Rightarrow O(n^2)$



```

for (i = 1; i <= n; i++) {
    for (j = 1; j <= i; j++) {
        printf("%d", i+j);
    }
}

```

Time complexity $\Rightarrow O(n^2)$

Ex:-

```

for (i = 1; i <= n; i++) {
    for (j = 1; j <= n; j = j * 2) {
        printf("%d", i+j);
    }
}

```

Time complexity $\Rightarrow O(n * \log n)$

$O(1) < O(\log n) < O(n) < O(n \cdot \log n) <$

$O(n^2) < O(n^3) < O(2^n)$

* $O(1)$ is the best case to check
best time complexity.

Q. To find time complexity of linear search.

A)

```

for(i=0; i < n; i++) {  $\Rightarrow O(n)$ 
    if (t == a[i])  $\Rightarrow O(1)$ 
    printf("Target found");  $\Rightarrow O(1)$ 
    break;  $\Rightarrow O(1)$ 
}
if(i == n)  $\Rightarrow O(1)$ 
printf("Target not found");  $\Rightarrow O(1)$ 

```

Total Time complexity $\Rightarrow O(n)$

* Time complexity for Binary Search is $\Rightarrow O(\log n)$

Q. Find time complexity of Bubble Sort.

```

for(p=1; p < n; p++) {  $O(n)$ 
    for(i=1; i < n-p; i++) {  $O(n \times n)$ 

```

* The time complexity of bubble sort is $O(n^2)$.

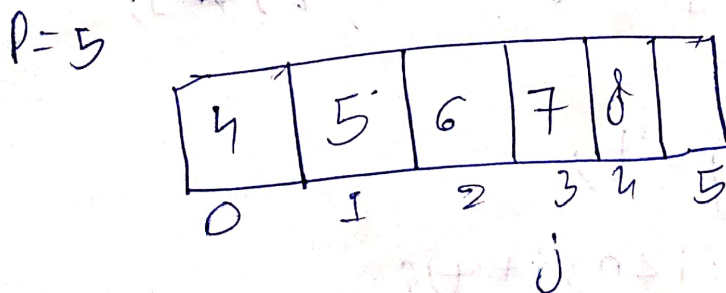
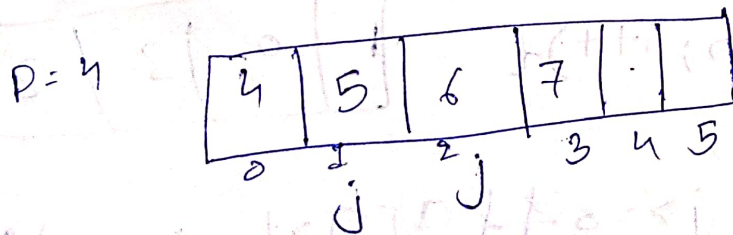
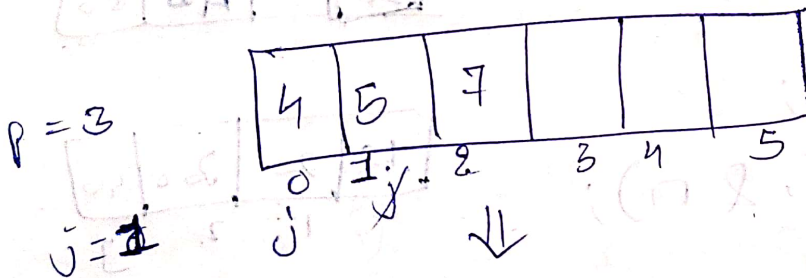
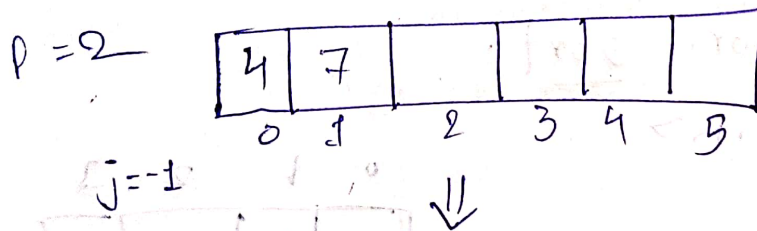
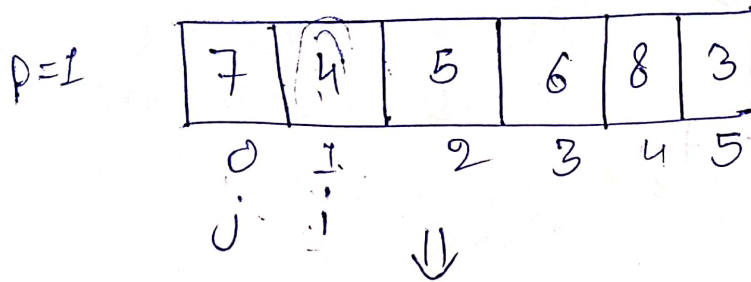
$\rightarrow \log_a y = y \log x$ $\rightarrow \log_a y = \log x + \log y$ $\rightarrow \log \log n = \log(\log n)$ $\rightarrow \log_a^x b = x \log_a b$ $\rightarrow \log n = \log_{10} n$	$\rightarrow \log_n^k = (\log n)^k$ $\rightarrow \log_y^x = \frac{\log x}{\log y}$ $\rightarrow \log_a^x = \frac{\log x}{\log a}$	$\rightarrow$ Sum of n natural numbers = $\frac{n(n+1)}{2}$ (Arithmetic). $\rightarrow$ Geometric series = $\frac{x^{n+1} - 1}{x - 1}$ $(1 + x + x^2 + \dots + x^n)$ $\rightarrow$ Harmonic series = $\log n$ $(1 + \frac{1}{2} + \dots + \frac{1}{n})$ $\rightarrow \sum_{k=1}^n \log k \approx n \log n$ $\rightarrow \sum_{k=1}^n k^p = 1^p + 2^p + \dots + n^p = \frac{1}{p+1} n^{p+1}$
--	---	---

Insertion Sort

24/12/24

→ In insertion sort a data can be placed into the direct location in the sorted list.

→ A data from unsorted list can be placed in correct location of sorted list.



```
for(i=1; i<n; i++)
```

```
{
    t = a[i]
```

```
    for (j=i-1; j>=0 && a[j]>t; j--) {
```

```
        a[j+1] = a[j];
```

```
    }
```

```
    a[j+1] = t;
```

```
}
```

Code for insertion sort.

```
#include <stdio.h>
```

```
int main ( )
```

```
{
```

```
    int n, i, j, t;
```

```
    scanf ("%d", &n);
```

```
    int a[n];
```

```
    for(i=0; i<n; i++) {
```

```
        t = a[i];
```

```
        for (j=i-1; j>=0 && a[j]>t; j--) {
```

```
            a[j+1] = a[j]; a[j+1] = a[j];
```

```
        }
```

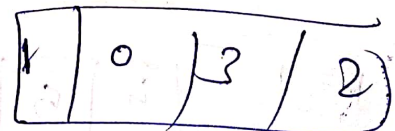
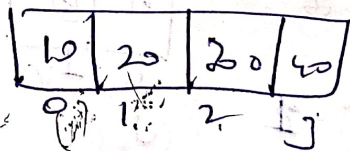
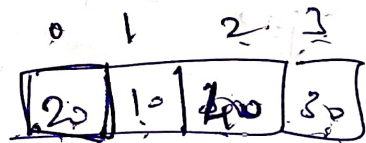
```
        a[j+1] = t;
```

```
    }
    for(i=0; i<n; i++) {
```

```
        printf ("%d", a[i]);
```

```
    }
```

```
    return 0;
```



$j = j+1$
 $2 \quad 4 \quad 6$
 $i = 0$
 $j = i-1$
 $2-1=1$

For printing each pass.

```
#include <stdio.h>
```

```
int main () {
```

```
int n, j, i, t;
```

```
scanf ("%d", &n);
```

```
int a[n];
```

```
for (i=0; i<n; i++) {
```

```
scanf ("%d", &a[i]);
```

```
}
```

```
for (i=0; i<n; i++)
```

```
{
```

```
t = a[i];
```

```
for (j=i-1; j>=0 & a[j]>t; j--)
```

```
{
```

```
a[j+1] = a[j];
```

```
for (i=0; i<n; i++) {
```

```
printf ("%d\t", a[i]);
```

```
printf ("\n");
```

```
}
```

```
}
```

```
a[j+1] = t;
```

```
}
```

```
for (i=0; i<n; i++) {
```

```
printf ("%d\t", a[i]);
```

```
return 0;
```

Code for insertion sort using functions

```
#include <stdio.h>
void InsertionSort (int A[], int n) {
    int i, j, t;
    for (i = 1; i < n; i++) {
        t = A[i];
        j = i - 1;
        while (A[j] > t & j >= 0) {
            A[j] = A[j + 1];
            j--;
        }
        A[j + 1] = t;
    }
}
// for (i = 0; i < n; i++)
int main() {
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }
    for (i = 0; i < n; i++) {
        printf("%d", A[i]);
    }
    return 0;
}
```

* Time complexity for insertion sort is $O(n^2)$. (Descending order).

* If the input is in sorted order then Insertion sort work $O(n)$ as best case.

* The average case time complexity is also $O(n^2)$.

Shell sort

→ Shell sort is created by Donald L. Shell.

→ The shell sort is modified version of insertion sort.

18 12 4 6 - 3 7 9 21 25 32 5

$$gap = \frac{n}{2}$$

$$= \frac{18}{2} = 9$$

18 12 4 6 - 3 7
 ↓ ↓ ↓ ↓ ↓ ↓
 9 21 25 32 5 7

9 12 4 6 - 3 - 7
 18 21 25 32 5 7

9 12 4 6 - 3 - 7 18 21 25 32 5

$$gap = \frac{6}{2} = 3$$

9 12 4 6 - 3 - 7

6 - 3 - 7 ⇒ 9 5 4

18 21 25 18 12 7
 32 5 7 32 21 25

6 - 3 - 7 9 5 4 18 12 7 32 21 25

$$\text{gap} = \frac{3}{2} = 1$$

6	-7
-3	-3
-7	4
9	5
5	6
4	7
18	9
12	12
7	18
32	21
21	25
25	32

Q. Take array size 'n' to apply shell sort for the following elements.

17 2 -6 -10 5 12 -21 18 3

$$\text{gap} = n/2 = 9/2 = 4$$

17	2	-6	-10	5	12	-21	18	3
5	12	-21	18	3	2	-21	-10	
3								

3 2 -21 -10 5 12 -6 18 17



$$\lambda/2 = \eta/2 = 2$$

$$\begin{array}{cc} 3 & 2 \\ -21 & -10 \end{array}$$

$$\begin{array}{cc} 5 & 12 \\ -6 & 18 \end{array}$$

$$17$$

$$-21$$

$$-6$$

$$3$$

$$5$$

$$17$$

$$-10$$

$$2$$

$$12$$

$$18$$

$$-21 \quad -10 \quad -6 \quad 2$$

$$3$$

$$12$$

$$9$$

$$18 \quad 17$$

$$\lambda/2 = 2/2 = 1$$

$$-21$$

$$-10$$

$$-6$$

$$2$$

$$3$$

$$12$$

$$5$$

$$18$$

$$17$$

$$\Rightarrow$$

$$-21$$

$$-10$$

$$-6$$

$$2$$

$$3$$

$$5$$

$$12$$

$$17$$

$$18$$

$$e = 5$$

$$T = \frac{h\nu}{e}$$

$$= 92$$

$$F_{\text{oc}} (1)$$

Code for shell sort

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, gap, j, temp;
```

```
    scanf ("%d", &n);
```

```
    int a[n];
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf ("%d", &a[i]);
```

```
    }
```

```
    for (gap = n/2; gap > 0; gap = gap/2) {
```

```
        for (i = gap; i < n; i++) {
```

```
            temp = a[i];
```

```
            for (j = i; j >= gap & a[j-gap] > temp; j = j - gap) {
```

```
                a[j] = a[j - gap];
```

```
            } a[j] = temp;
```

```
        }
```

```
    }
```

```
    for (i = 0; i < n; i++) {
```

```
        printf ("%d", a[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

Time Complexity of Shell Sort

Best case:- $O(n) \times O(\log n)$
(sorted order) $\Rightarrow O(n \log n)$.

* In the shell sort the gap will work for $O(\log n)$ time & insertion sort will work for $O(n)$ time with the best case.

Worst case:- $O(\log n) \times O(n^2)$

* The worst case of shell sort is $O(n^2)$.

Average case:- (cannot be predicted by using inputs.)

→ But usually it is $O(n \log n)$ or $O(n^2)$.

Quick Sort (Partition exchange sort).

- The quick sort is following the divide and conquer method.
- Any element can be taken as pivot element.

Step 1:- Fix the lower bound & upper bound & pivot indices

Step 2:- Place the pivot index element $a[p]$ in the array and then
 $a[1 \dots p-1] < a[p] < a[p+1 \dots h]$

Algorithm (Two-point Algorithm)

Step 1:- Initialize $i = l$ and $j = h$

Step 2:- If $l < h$ to apply quick sort.

Step 3:- Repeatedly check $a[i] \leq a[p]$ then
 $i = i + 1;$

Step 4:- Repeatedly check $a[j] > a[p]$
then $j = j - 1;$

Step 5:- If $i < j$ to swap $a[i]$ and $a[j]$

Step 5.1:- Repeat step no. 3 to 5

Step 6:- If $i > j$ swap $a[j]$ and pivot $a[p]$.

Step 7:- Divide the array as $(a, i-1)$ and $(a, i+1, h);$

12	4	3	6	18	14	9	13	17
----	---	---	---	----	----	---	----	----

$n = 9$

i j P

$$i=0 \quad a[i] \leq a[P]$$

$$12 \leq 12$$

$$i=1 \quad 4 \leq 12$$

$$i=2 \quad 3 \leq 12$$

$$i=3 \quad 6 \leq 12$$

$$i=4 \quad 18 \leq 12$$

$$a[j] > a[P]$$

$$j=8 \quad 17 > 12$$

$$j=7 \quad 13 > 12$$

$$j=6 \quad 9 > 12$$

12	4	3	6	9	14	18	13	17
----	---	---	---	---	----	----	----	----

9	4	3	6	12	14	18	13	17
---	---	---	---	----	----	----	----	----

$$i=4 \quad 12 \leq 12 \quad \checkmark$$

$$i=5 \quad 14 \leq 12 \quad \times$$

$$j=6 \quad 18 > 12 \quad \checkmark$$

$$j=5 \quad 14 > 12 \quad \checkmark$$

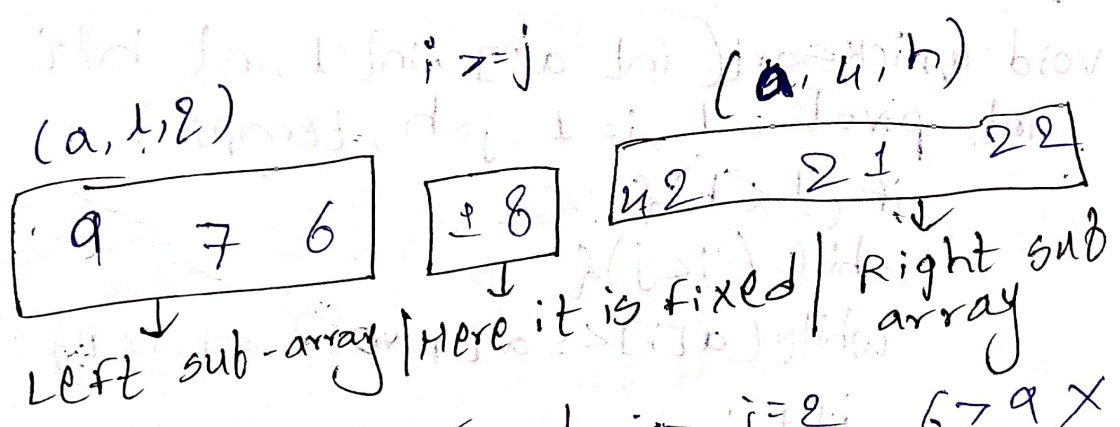
$$j=4 \quad 9 > 12 \quad \times$$

18 7 6 21 42 9 22
l=0 j=6 h=6
i=0
p=0
l < h
0 < 6

i=0	18 <= 18 ✓	j=6	22 > 18 ✓
i=1	7 <= 18 ✓	j=5	9 > 18 X
i=2	6 <= 18 ✓		
i=3	21 <= 18 X		

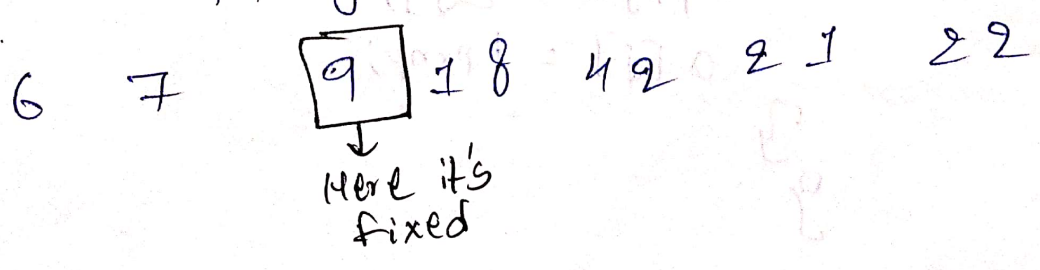
18 7 6 9 42 21 22

i=3	9 <= 18 ✓	j=5	21 > 18 ✓
i=4	42 <= 18 X	j=4	42 > 18 ✓
		j=3	9 > 18 X



i=0	9 <= 9 ✓	j=2	6 > 9 X
i=1	7 <= 9 ✓		
i=2	6 <= 9 ✓		

i = 2 = j
i >= j then swap a[i] & a[j]



$i = 4$ $42 < 42$ ✓
 $i = 5$ $21 < 42$ ✓
 $i = 6$ $22 < 42$ ✓

$j = 6$ $22 > 42$

$i \geq j$
 $i = j = 6$ so,
 $\text{swap}(a[i], a[j])$

6 7 9 18 2 2 21 42

~~i = 6~~ ~~42~~

Code for quick sort
`#include <stdio.h>`

`void quicksort(int a[], int l, int h) {`
`int pivot = l, i = l, j = h, temp;`
`if (l < h) {`
`while (i < j) {`
`while (a[i] <= a[pivot] && i < h)`
`i++;`
`while (a[j] > a[pivot])`
`j--;`
`if (i < j)`
`temp = a[i];`
`a[i] = a[j];`
`a[j] = temp;`
`}`
`}`


```

temp = a[i]
a[i] = a[pivot];
a[pivot] = temp;
quickSort(a, l, j-1);
quickSort(a, j+1, h);
}

```

```

int main() {
    int n;
    scanf("%d", &n);
    int a[n];
    for(i=0; i<n; i++) {
        scanf("%d", &a[i]);
    }
    quickSort(a, 0, n-1);
    for(i=0; i<n; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}

```

Time complexity for quick sort

Worst-case:- If the inputs are in sorted order the quicksort time complexity is $T(n) = T(n-1) + C * n$

$$T(n-1) = T(n-2) + C * (n-1)$$

$$T(n-2) = T(n-3) + C * (n-2)$$

$$T(n-3) = T(n-4) + C * (n-3)$$

constant time to merge element.

$$T(n) = T(n-2) + C * (n-1) + C * n$$

$$= T(n-3) + C * (n-2) + C * (n-1) + C * n$$

$$T(n) = T(n-3) + C * n - 2 * C + C * n - C + C * n$$

$$= T(n-3) + 3 * C * n - C - 2C$$

$$T(n) = T(n-k) + k * C * n - (C + 2C + 3C + \dots + kC)$$

$$k=n \Rightarrow T(0) + C * n^2 - C(1+2+3+\dots+n)$$

$$= C * n^2 - C * n * (n+1)/2$$

$$Cn^2 - \frac{Cn^2}{2} - \frac{Cn}{2} = O(n^2)$$

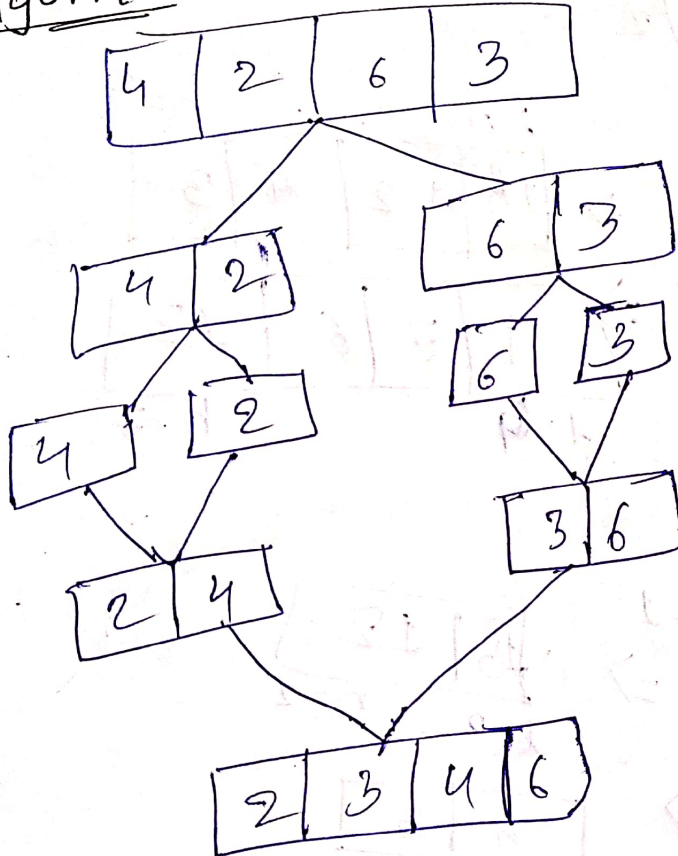
Best case:- $O(n \log n)$.

Average case:- $O(n \log n)$.

merge sort

- It is having least time complexity than any other sorting techniques.
- It follows divide & conquer technique.
- The fastest sorting technique.

Algorithm



Array - divide methods

```
void divide (int arr[], int l, int h) {
```

```
    if (l < h) {
```

```
        int m = l + (h - l) / 2;
```

```
        divide (arr, l, m); // left sub array
```

```
        divide (arr, m + 1, h); // right sub array
```

```
        conquer (arr, l, m, h);
```

```
    }
```

```
}
```


$n = 8$
 elements:

18	12	4	3	9	7	6	13
----	----	---	---	---	---	---	----

 $l = 0$ $m = j = m+1$ $h = 7$

$l < h \Rightarrow 0 < 7 \checkmark$

$m = \lfloor (l + h - 1) / 2 \rfloor$
 $= \lfloor (0 + 7 - 0) / 2 \rfloor$

$m = 3$

divide(a, 0, 3); $\Rightarrow$

18	12	4	3
----	----	---	---

divide(a, 4, 7); $\Rightarrow$

9	7	6	13
---	---	---	----

$l < h$

$0 < 3$
 $m = \lfloor (3 - 0) / 2 \rfloor = 1$

divide(a, 0, 1) $\Rightarrow$

18	12
----	----

divide(a, 2, 3) $\Rightarrow$

4	3
---	---

$4 < 7$

$m = \lfloor (7 - 4) / 2 \rfloor$
 $= 4 + 1$

$m = 5$

divide(a, 4, 5) $\Rightarrow$

9	7
---	---

divide(a, 6, 7) $\Rightarrow$

6	13
---	----

$$0 < 1$$

$$m = 0 + (1-0)/2 = 0$$

$$\text{divide}(a, 0, 0) \Rightarrow \boxed{18} \quad l=0, h=0$$

$$\boxed{12} \quad \text{divide}(a, 1, 1) \quad l=1, h=1$$

$$2 < 3$$

$$m = 2 + (3-2)/2$$

$$\boxed{m=2}$$

$$\text{divide}(a, 2, 2) \Rightarrow$$

$$\boxed{4} \quad l=2, h=2$$

$$\text{divide}(a, 3, 3) \Rightarrow \boxed{3} \quad l=3, h=3$$

$$4 < 5$$

$$mid = 4 + (5-4)/2$$

$$mid = 4$$

$$\text{divide}(a, 4, 4) \Rightarrow$$

$$\boxed{9} \quad l=4, h=4$$

$$\text{divide}(a, 5, 5) \Rightarrow$$

$$\boxed{7} \quad l=5, h=5$$

$$6 < 7$$

$$m = 6 + (7-6)/2$$

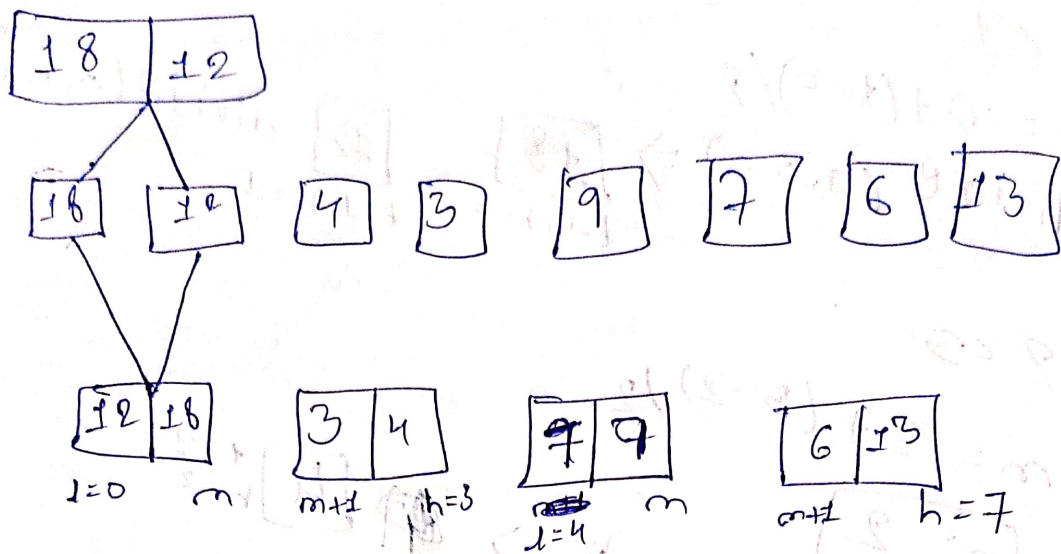
$$m = 6$$

$$\text{divide}(a, 6, 6) \Rightarrow$$

$$\boxed{6} \quad l=6, h=6$$

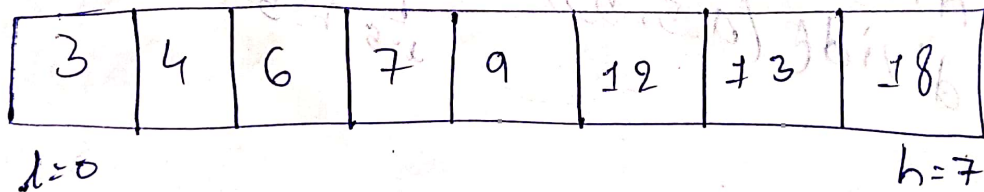
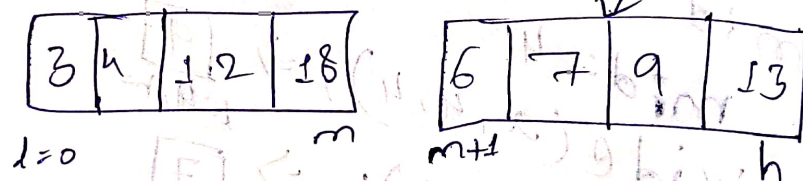
$$\text{divide}(a, 7, 7) \Rightarrow$$

$$\boxed{3} \quad l=7, h=7$$



$i \leq m$ & $j \leq h$
 if ($a[i] < a[j]$)
 $t[i] = a[i++];$
 else
 $t[j] = a[j++];$

if we perform this logic we get these arrays



Code for Merge Sort

```

#include <stdio.h>
void conquer (int a[], int l, int m, int h) {
    void divide (int a[], int l, int h) {
        if (l < h) {
            int m = l + (h - l) / 2;
            divide (a, l, m);
            divide (a, m+1, h);
            conquer (a, l, m, h);
        }
    }
    (inside the function)
    int i = l, j = m+1;
    while (i <= m & j <= h)
        if (a[i] < a[j])
            t[i] = a[i++];
    else
        t[j] = a[j++];
}

```



```
}  
while (i <= m)
```

```
    t[i] = a[i++];
```

```
while (j <= n)
```

```
    t[j] = a[j++];
```

```
#include <stdio.h>
```

```
void conquer (int a[], int l, int h) {
```

```
    int i = l; j = m+1; t[h+1] = 0;
```

```
    while (i <= m & j <= n) {
```

```
        if (a[i] < a[j])
```

```
            t[k++] = a[i++];
```

```
        else
```

```
            t[k++] = a[j++];
```

```
    }  
    while (i <= m)
```

```
        t[k++] = a[i++];
```

```
    while (j <= n)
```

```
        t[k++] = a[j++];
```

```
    k = 0;
```

```
    for (i = l; i <= h; i++)
```

```
        a[i] = t[k++];
```

```
}
```

```
void divide (int a[], int l, int h) {
```

```
    if (l < h) {
```

```
        int m = l + (h - l) / 2;
```

```
        divide (a, l, m);
```

```
        divide (a, m+1, h);
```

```
        conquer (a, l, m, h);
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int a[n], i;
```

```
    for(i = 0; i < n; i++) {
```

```
        scanf("%d", &a[i]);
```

```
        divide(a, 0, n-1);
```

```
    } for(i = 0; i < n; i++) {
```

```
        printf("%d", a[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

Time complexity $\Rightarrow$

$$T(n) = T(n/2) + T(n/2) + C \cdot n$$

$$= 2 \cdot T(n/2) + C \cdot n$$

$$T(n/2) = T(n/4) + T(n/4) + C \cdot n/2$$

$$= 2 \cdot T(n/4) + C \cdot n/2$$

$$T(n) = 4 \cdot T(n/4) + 2 \cdot C \cdot n/2 + C \cdot n$$

$$= 4 \cdot T(n/4) + 2 \cdot C \cdot n$$

$$T(n) = 2^k \cdot T(n/2^k) + k \cdot C \cdot n$$

$$n/2^k = 1$$

$$n = 2^k \Rightarrow \log n = k \Rightarrow T(1) + \log n \cdot C \cdot n$$

nt cologn

 $O(n \log n)$

This is the time complexity for all the three ~~cases~~ cases: best & Average cases.

Time Complexity.

	Best	Avg	Worst
Sorting			
Insertion	$O(n^2)$	$O(n^2)$	$O(n^2)$
Shell	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

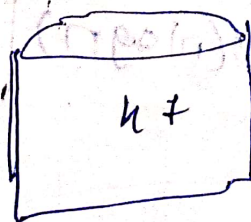
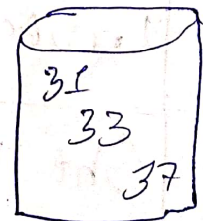
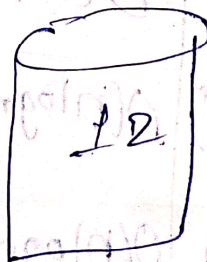
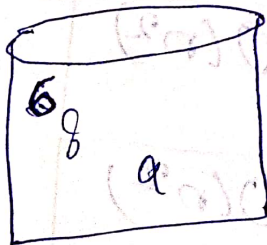
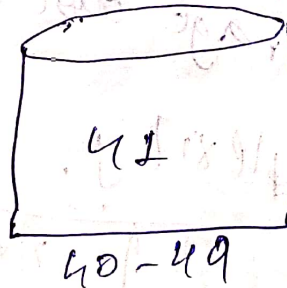
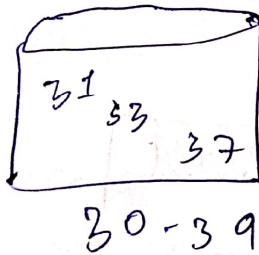
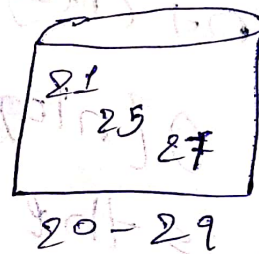
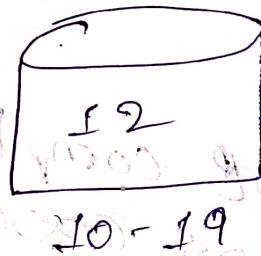
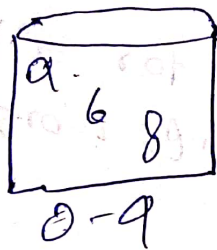
Bucket Sort

06/04/

In a bucket sort the elements are uniformly distributed groups or buckets then sort each one of every bucket using any other sort and then merge the group or bucket into given array

Ex:-

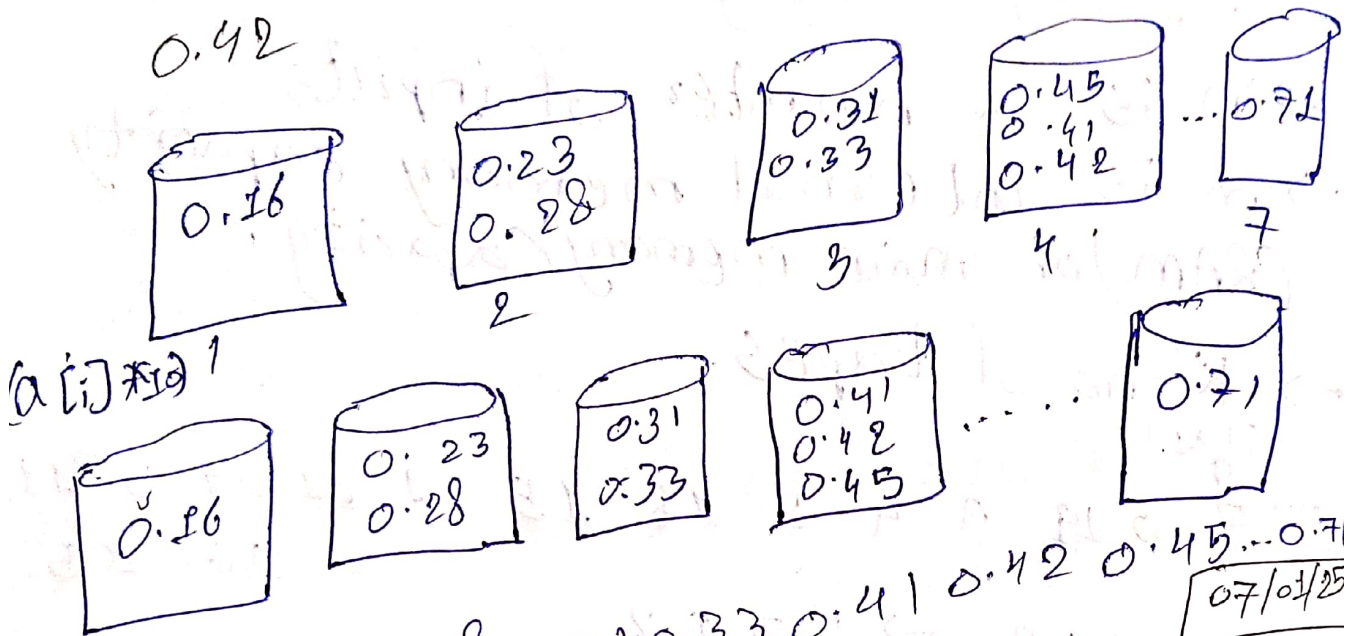
41 21 25 9 6 12 33 37 31 23 27 8



merge $\Rightarrow$ 6 8 9 12 21 25 27 31 33 37 41

Ex: 0.45 0.41 0.23 0.16 0.31 0.28 0.33 0.3

0.42



Time Complexity of Bucket sort

Best Case: $O(n+k) \Rightarrow$ (Average case).

Worst Case: $O(n^2)$

External Sorting

By using external sorting we can sort when large amount of data is present in the file.

Types of External Sorting

- ① Two-way sort
- ② Multi-way sort
- ③ PolyPhase Sort

Two-way Sorting

- 'n' is a number of inputs.
- 'm' is internal memory capacity (RAM) or main memory capacity.
- 'K' no of tapes.

Ex: ^{Tape} 7 3 19 9 4 2 18 15 21 ~~17~~ 5 1
16 13

7, 3, 12 → 3, 7, 12

T_{b1} = 3, 7, 12 → 1st run

9, 4, 2 → 2, 4, 9 - ~~1st run~~

T_{b2} = 2, 4, 9 → 1st run

18, 15, 21

T_{b1} = 3, 7, 12 | 18, 15, 21 → 2nd run

~~20~~ 17 5 11

T_{b2} = 5, 11, 17

= 2, 4, 9 | 5, 11, 17 → 2nd run

16, 13

T_{b1} = 3, 7, 12 | 15, 18, 21 | 13, 16 → 3rd run

Merging

Pass no = 1

T_{b1} = 3 7 12 | 15, 18, 21 | 13, 16
" " 9 | 5, 11, 17

$T_{a1} = 2, 3, 4, 7, 9, 12, 15, 16$

$T_{a2} = 5, 11, 15, 17, 18, 21$

Pass no = 2

$T_{b1} = 2, 3, 4, 5, 7, 9, 11, 12, 15, 17, 18, 21$

Pass no = 3

$T_{a1} = 2, 3, 4, 5, 7, 9, 11, 12, 13, 15, 16, 17, 18, 21$

For nula for find passes = $\log(n/m) + 1$

$$\log(14/3) + 1$$

$$\log(2^3) + 1$$

$$2 \log 2 + 1$$

$$2 + 1$$

$$= 3$$

Ex:- $N=17, m=4, K=2.$

$T_{a1} = 12 \ 9 \ 6 \ 21 \ 42 \ 5 \ 11 \ 18 \ 35 \ 3$
 $49 \ 29 \ 19 \ 15 \ 25 \ 27 \ 7.$

$T_{b1} = 12 \ 9 \ 6 \ 21$

$T_{b1} = 5 \ 9 \ 12 \ 21 \rightarrow 1st \ run$

$T_{b2} = 42 \ 5 \ 11 \ 18$

$T_{b2} = 5 \ 11 \ 18 \ 42 \rightarrow 1st \ run$

$T_{b1} = 35 \ 3 \ 49 \ 29$

$T_{b1} = 3 \ 29 \ 35 \ 49 \rightarrow 2nd \ run$

$$T_{b2} = 19 \ 15 \ 25 \ 27$$

$$T_{b2} = 15 \ 19 \ 25 \ 27 \rightarrow 2nd \ run$$

$$T_{b2} = 7 \rightarrow 3rd \ run$$

$$T_{b2} = 6 \ 9 \ 12 \ 21 \mid 3 \ 29 \ 35 \ 49 \mid 7$$

$$T_{b2} = 5 \ 11 \ 18 \ 42 \mid 15 \ 19 \ 25 \ 27$$

merging

$$T_{a1} = 5 \ 6 \ 9 \ 11 \ 12 \ 18 \ 21 \ 42 \mid 7$$

$$T_{a2} = 3 \ 15 \ 19 \ 29 \ 25 \ 35 \ 27 \ 49$$

$$P=2$$

$$T_{a1} = 3 \ 5 \ 6 \ 9 \ 11 \ 12 \ 15 \ 18 \ 19 \ 21 \ 27 \ 29 \ 35 \ 49$$

$$T_{a2} = 7$$

$$P=3$$

$$T_{a1} = 3 \ 5 \ 6 \ 7 \ 9 \ 11 \ 12 \ 15 \ 18 \ 19 \ 21 \ 27 \ 29 \ 35 \ 49$$

$$\log(n/m) + \log(17/4) + 1$$

$$\log(4) + 1$$

$$\log(2^2) + 1$$

$$2 \log 2 + 1$$

$$2 + 1$$

$$= 3$$